

Ground Robotics Kinematics: A Generalized Framework for Forward and Inverse Kinematics of Wheeled Terrestrial Robots

Juan Francisco Rascón Crespo

Friday 13th March, 2026

Abstract

This document presents a rigorous analytical development of the forward and inverse kinematics for wheeled terrestrial robots. Starting from the definition of an invariant base coordinate system for each wheel, a system of matrix equations is formulated that relates the individual velocities to the chassis's state of motion. Given that platforms with multiple wheels typically generate overdetermined systems, their optimal resolution is addressed using the least squares method and the Moore-Penrose pseudoinverse. The text places a fundamental emphasis on software implementation and numerical stability, comprehensively justifying the use of Singular Value Decomposition (SVD) to bypass algebraic instabilities. Furthermore, the condition number (κ) analysis is introduced as a critical diagnostic tool to distinguish between the computational robustness of the algorithm and the physical viability of the mechanical design. Finally, this generalized theoretical framework is applied to the kinematic formulation and resolution of various standard topologies in mobile robotics, ranging from differential drive and Ackermann steering systems to omnidirectional platforms with Mecanum wheels and *swerve drive* configurations.

1 Motion constraints imposed by the wheels

In a terrestrial robot, the wheels impose motion constraints. In conventional wheels, whether steerable or non-steerable, as the wheel rotates about its axis of rotation, it moves following a straight path in a specific direction called the **rolling direction**. In non-steerable conventional wheels, the rolling direction is fixed, always pointing in the same direction, since the wheel cannot change its orientation relative to the chassis. In steerable conventional wheels, the wheel first orients itself toward the desired rolling direction (pivoting on its vertical axis) and then rotates about its axis of rotation to move in that direction. Both steerable and non-steerable con-

ventional wheels cannot move in the direction perpendicular to the rolling direction, which is the direction of their rotation axis. In other words, the lateral velocity of the wheel is zero, which is known as the **no-lateral-slip constraint**.

Conversely, in omnidirectional and Mecanum/Swedish wheels, due to the presence of free-spinning rollers on their periphery, the rolling direction of the main wheel, although fixed and unchanging over time, does not necessarily have to coincide with the actual direction of the wheel's movement on the ground. The passive rotation of the rollers allows the wheel to have a non-zero lateral velocity (in a direction perpendicular to the rolling direction; *i.e.*, the direction of the wheel's axis of rotation), eliminating the no-lateral-slip constraint.

These constraints can be expressed mathematically as equations that relate the linear and angular velocity of the wheel to the robot's velocity. These equations are fundamental for the design of controllers and motion planners for terrestrial robots, as they determine how the robot can move in its environment. By stacking these constraints for each wheel, the forward and inverse kinematics equations can be derived for different terrestrial robot configurations, such as differential drive robots, omnidirectional wheeled robots, and steerable active wheeled robots (swerve drive), among others.

Figure 1 shows the 2D positioning of a wheel relative to the robot's coordinate system (CS), $\mathcal{R}$. The term α represents the angle formed by the $X_{\mathcal{R}}$ axis and the wheel's position vector, $\mathbf{d}$, and due to how the wheel is *initially* positioned, it is also the angle formed by the $X_{\mathcal{R}}$ axis and the $Y_{\mathcal{W}}$ axis. The $X_{\mathcal{W}}$ axis is perpendicular to the $Y_{\mathcal{W}}$ axis and its orientation with respect to the $X_{\mathcal{R}}$ axis is $\psi = \alpha - \frac{\pi}{2}$. The magnitude of the vector $\mathbf{d}$ is the distance between the robot's CS and the wheel's CS, $|\mathbf{d}| = d$.

$$\mathbf{d} = \begin{bmatrix} d \cos \alpha \\ d \sin \alpha \\ 0 \end{bmatrix} \quad (1)$$

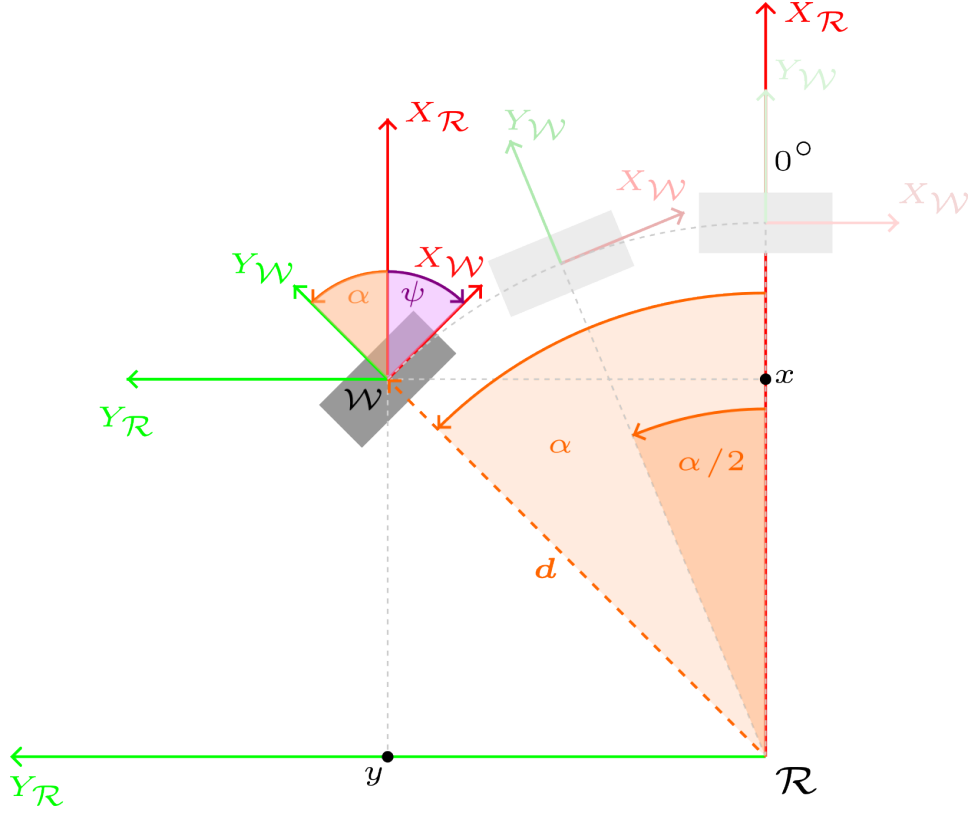


Figure 1: 2D positioning of a wheel relative to the robot's CS, $\mathcal{R}$.

Next, an angle β is considered, which allows the wheel to be oriented in the desired manner with respect to the robot's CS, $\mathcal{R}$ (Figure 2), giving rise to the **wheel's base CS**, $\mathcal{B}$. The CS $\mathcal{B}$ maintains a constant position and orientation over time with respect to the CS $\mathcal{R}$ and is *conveniently suitable* for defining the wheel's orientation during the vehicle's movement and applying the motion constraints. The final orientation of the wheel is given by the angle ψ , formed between the $X_{\mathcal{R}}$ axis and the $X_{\mathcal{B}}$ axis.

$$\psi = \alpha + \beta - \frac{\pi}{2}$$

And, to simplify the notation, the symbol ϕ is introduced as:

$$\phi = \alpha + \beta, \quad \psi = \phi - \frac{\pi}{2}$$

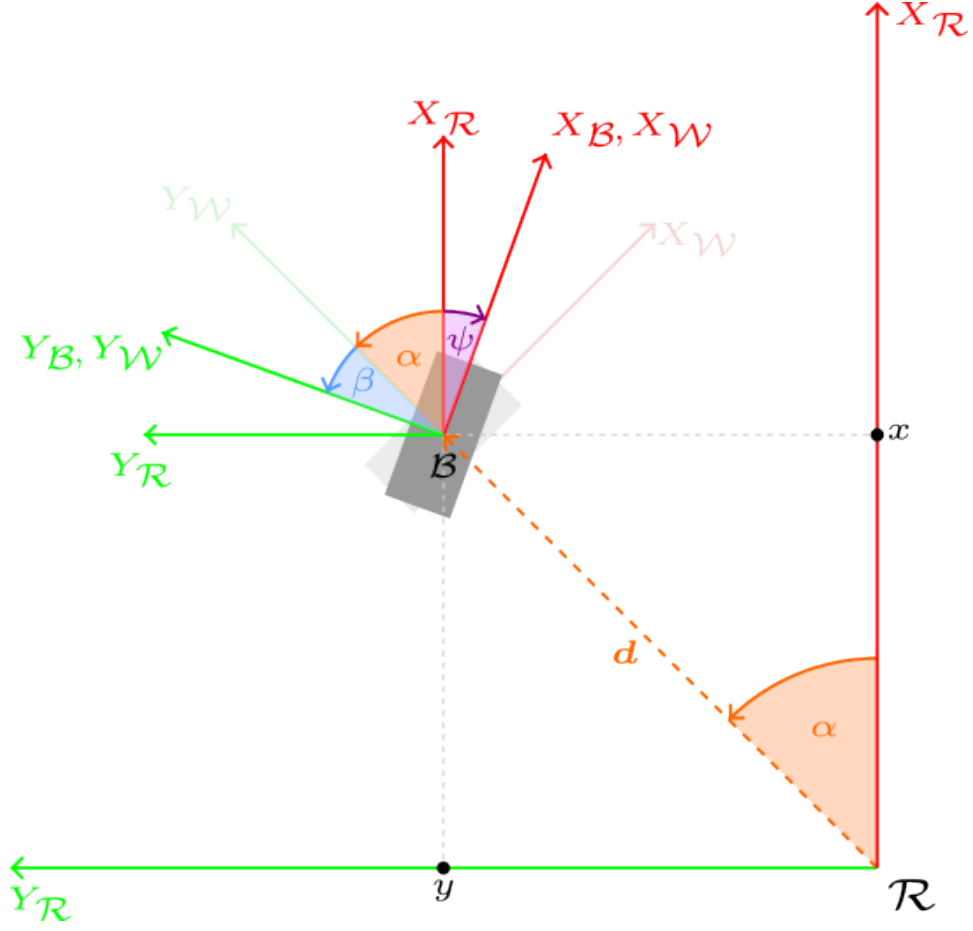


Figure 2: Orientation of the wheel in the direction of interest, through the angle β .

In non-steerable conventional wheels, the $X_{\mathcal{B}}$ axis and the X axis fixed to the wheel, $X_{\mathcal{W}}$, which indicates the wheel's rolling direction, coincide all the time. Just for the sake of completeness, it is worth mentioning that in this type of wheels, the $Y_{\mathcal{B}}$ axis and the Y axis fixed to the wheel, $Y_{\mathcal{W}}$, which indicates the direction of its rotation axis, obviously also coincide all the time. In steerable wheels, the CS fixed to the wheel, $\mathcal{W}$, changes its orientation (angle θ) with respect to the CS $\mathcal{B}$; that is, only when $\theta = 0$ do both CSs coincide. See Figure 3.

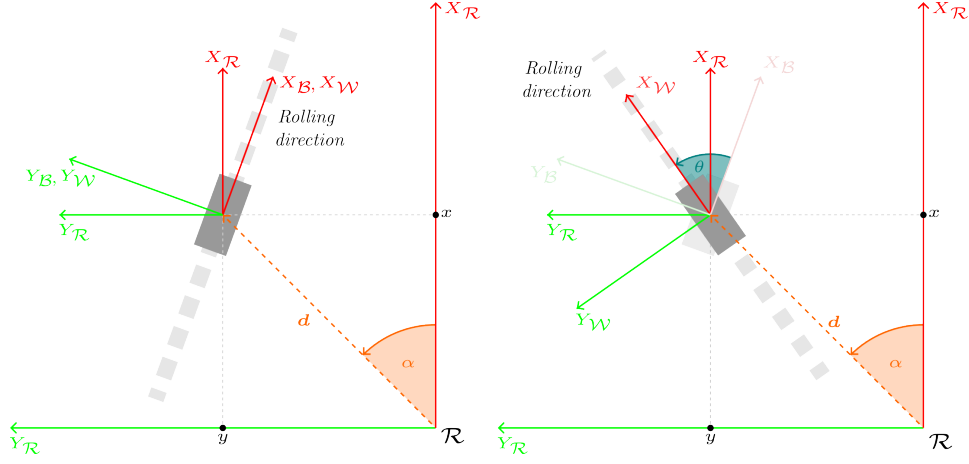


Figure 3: Left: Non-steerable conventional wheel, with fixed rolling direction. Right: Steerable conventional wheel, with variable rolling direction, achieved by pivoting the wheel by an angle θ relative to the wheel's base CS, $\mathcal{B}$.

The linear velocity of the base, expressed in the robot's CS, is denoted as:

$$\mathcal{R}\mathbf{V}_r = \begin{bmatrix} \mathcal{R}V_{r,x} \\ \mathcal{R}V_{r,y} \\ 0 \end{bmatrix} \quad (2)$$

and is expressed in m/s. The angular velocity of the base is denoted as:

$$\boldsymbol{\omega}_r = \begin{bmatrix} 0 \\ 0 \\ \omega_{r,z} \end{bmatrix} \quad (3)$$

and is expressed in rad/s.

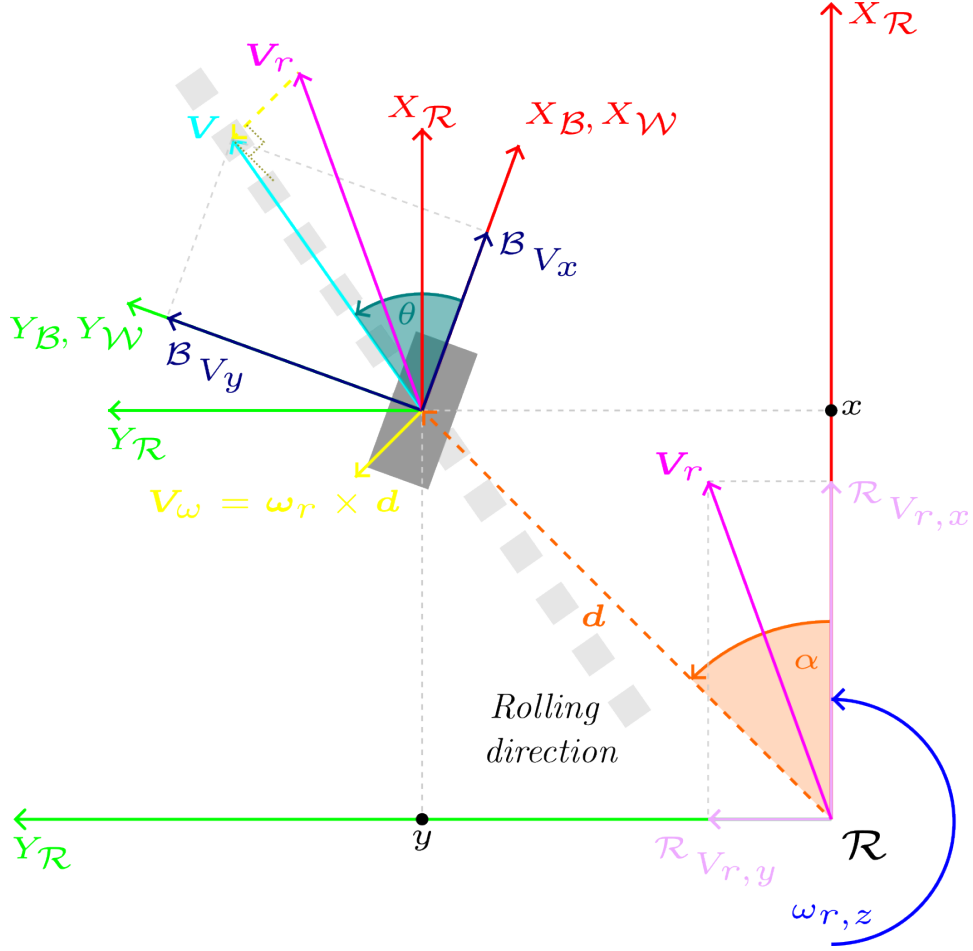


Figure 4: $\mathbf{V}_r$ is the robot's velocity vector. $\mathbf{V}_\omega$ is the tangential velocity vector of the wheel due to chassis rotation. $\mathbf{V}$ is the velocity vector of the steerable wheel.

The linear velocity of each wheel i , expressed in the CS $\mathcal{R}$, is denoted as:

$$\mathcal{R}\mathbf{V}_i = \mathcal{R}\mathbf{V}_r + \boldsymbol{\omega}_r \times \mathcal{R}\mathbf{d}_i = \mathcal{R}\mathbf{V}_r + [\boldsymbol{\omega}_r]_\times \mathcal{R}\mathbf{d}_i \quad (4)$$

where $\mathcal{R}\mathbf{d}_i$ is the position vector of wheel i relative to the CS $\mathcal{R}$, and $[\boldsymbol{\omega}_r]_\times$ is the skew-symmetric operator representing the cross product with $\boldsymbol{\omega}_r$. The cross product between $\boldsymbol{\omega}_r$ and $\mathcal{R}\mathbf{d}_i$ can be written as a matrix product using the skew-symmetric operator $[\cdot]_\times$: $\boldsymbol{\omega}_r \times \mathcal{R}\mathbf{d}_i = [\boldsymbol{\omega}_r]_\times \mathcal{R}\mathbf{d}_i$, and $[\boldsymbol{\omega}_r]_\times$ is defined as:

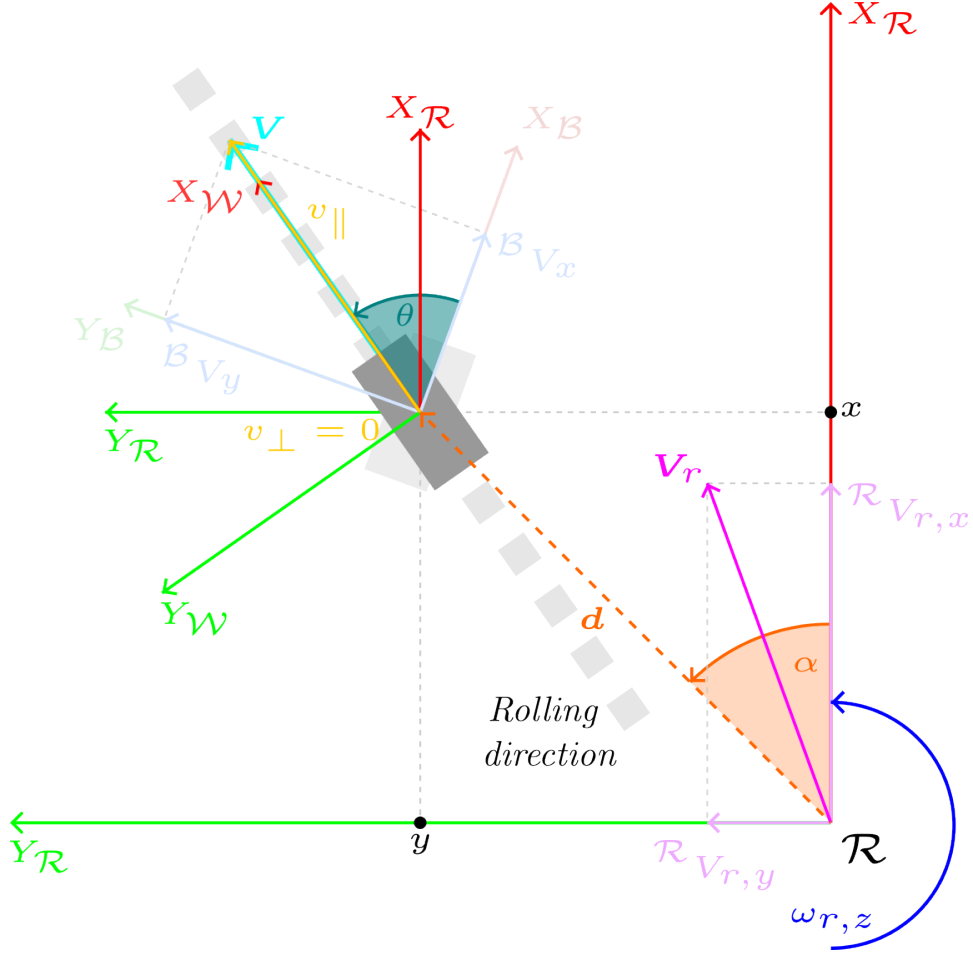


Figure 5: Relationship between the components of the wheel's velocity vector $\mathbf{V}$ in the CS $\mathcal{B}$, ${}^{\mathcal{B}}V_x$ and ${}^{\mathcal{B}}V_y$, and in the CS $\mathcal{W}$, $v_{\parallel}$ and $v_{\perp}$.

$$[\boldsymbol{\omega}_r]_{\times} \triangleq \begin{bmatrix} 0 & -\omega_{r,z} & \omega_{r,y} \\ \omega_{r,z} & 0 & -\omega_{r,x} \\ -\omega_{r,y} & \omega_{r,x} & 0 \end{bmatrix} \quad (5)$$

$$\begin{aligned} \boldsymbol{\omega}_r \times {}^{\mathcal{R}}\mathbf{d}_i &= [\boldsymbol{\omega}_r]_{\times} {}^{\mathcal{R}}\mathbf{d}_i \\ &= \begin{bmatrix} 0 & -\omega_{r,z} & 0 \\ \omega_{r,z} & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} d_i \cos \alpha_i \\ d_i \sin \alpha_i \\ 0 \end{bmatrix} \\ &= \begin{bmatrix} -\omega_{r,z} d_i \sin \alpha_i \\ \omega_{r,z} d_i \cos \alpha_i \\ 0 \end{bmatrix} \end{aligned} \quad (6)$$

$$\begin{aligned}
{}^{\mathcal{R}}\mathbf{V}_i &= {}^{\mathcal{R}}\mathbf{V}_r + \boldsymbol{\omega}_r \times {}^{\mathcal{R}}\mathbf{d}_i = \begin{bmatrix} {}^{\mathcal{R}}V_{r,x} - \omega_{r,z}d_i \sin \alpha_i \\ {}^{\mathcal{R}}V_{r,y} + \omega_{r,z}d_i \cos \alpha_i \end{bmatrix} \\
&= \begin{bmatrix} 1 & 0 & -d_i \sin \alpha_i \\ 0 & 1 & d_i \cos \alpha_i \end{bmatrix} \begin{bmatrix} {}^{\mathcal{R}}V_{r,x} \\ {}^{\mathcal{R}}V_{r,y} \\ \omega_{r,z} \end{bmatrix} \quad (7)
\end{aligned}$$

The vector ${}^{\mathcal{R}}\mathbf{V}_i$ represents the linear velocity of wheel i expressed in the CS $\mathcal{R}$. To apply the motion constraints to each wheel, its velocity must be expressed in its base CS, $\mathcal{B}_i$, ${}^{\mathcal{B}_i}\mathbf{V}_i$.

$${}^{\mathcal{B}_i}\mathbf{V}_i = {}^{\mathcal{B}_i}R(\psi_i)_{\mathcal{R}} {}^{\mathcal{R}}\mathbf{V}_i \quad (8)$$

where the rotation matrix ${}^{\mathcal{B}_i}R(\psi_i)_{\mathcal{R}}$ represents the orientation of the CS $\mathcal{R}$ in the CS $\mathcal{B}_i$, and is used to convert vectors expressed in the CS $\mathcal{R}$ to the base CS of wheel i , $\mathcal{B}_i$. The matrix known a priori is:

$${}^{\mathcal{R}}R(\psi_i)_{\mathcal{B}_i} = \begin{bmatrix} \cos \psi_i & -\sin \psi_i \\ \sin \psi_i & \cos \psi_i \end{bmatrix} \quad (9)$$

with $\psi_i = \phi_i - \frac{\pi}{2} = \alpha_i + \beta_i - \frac{\pi}{2}$, representing the orientation of the CS $\mathcal{B}_i$ relative to the CS $\mathcal{R}$. Since rotation matrices are orthonormal, their inverse is equal to their transpose, thus:

$${}^{\mathcal{B}_i}R(\psi_i)_{\mathcal{R}} = \left({}^{\mathcal{R}}R(\psi_i)_{\mathcal{B}_i}\right)^{-1} = \left({}^{\mathcal{R}}R(\psi_i)_{\mathcal{B}_i}\right)^{\top} = \begin{bmatrix} \cos \psi_i & \sin \psi_i \\ -\sin \psi_i & \cos \psi_i \end{bmatrix} \quad (10)$$

Next, the following trigonometric identities are applied to the matrix ${}^{\mathcal{B}_i}R(\psi_i)_{\mathcal{R}}$:

$$\begin{aligned}
\cos \psi_i &= \cos \left(\phi_i - \frac{\pi}{2} \right) = \sin \phi_i \\
\sin \psi_i &= \sin \left(\phi_i - \frac{\pi}{2} \right) = -\cos \phi_i
\end{aligned}$$

finally yielding:

$${}^{\mathcal{B}_i}R(\phi_i)_{\mathcal{R}} = \begin{bmatrix} \sin \phi_i & -\cos \phi_i \\ \cos \phi_i & \sin \phi_i \end{bmatrix} \quad (11)$$

$$\begin{aligned}
\mathcal{B}_i \mathbf{V}_i &= \begin{bmatrix} \mathcal{B}_i V_{i,x} \\ \mathcal{B}_i V_{i,y} \end{bmatrix} = \mathcal{B}_i R(\phi_i) \mathcal{R} \mathbf{V}_i \\
&= \begin{bmatrix} \sin \phi_i & -\cos \phi_i \\ \cos \phi_i & \sin \phi_i \end{bmatrix} \begin{bmatrix} 1 & 0 & -d_i \sin \alpha_i \\ 0 & 1 & d_i \cos \alpha_i \end{bmatrix} \begin{bmatrix} \mathcal{R} V_{r,x} \\ \mathcal{R} V_{r,y} \\ \omega_{r,z} \end{bmatrix} \quad (12)
\end{aligned}$$

$$\begin{bmatrix} \mathcal{B}_i V_{i,x} \\ \mathcal{B}_i V_{i,y} \end{bmatrix} = \begin{bmatrix} \sin \phi_i & -\cos \phi_i \\ \cos \phi_i & \sin \phi_i \end{bmatrix} \begin{bmatrix} 1 & 0 & -d_i \sin \alpha_i \\ 0 & 1 & d_i \cos \alpha_i \end{bmatrix} \begin{bmatrix} \mathcal{R} V_{r,x} \\ \mathcal{R} V_{r,y} \\ \omega_{r,z} \end{bmatrix} \quad (13)$$

$$\begin{bmatrix} \mathcal{B}_i V_{i,x} \\ \mathcal{B}_i V_{i,y} \end{bmatrix} = \begin{bmatrix} \sin \phi_i & -\cos \phi_i & -d_i (\sin \alpha_i \sin \phi_i + \cos \alpha_i \cos \phi_i) \\ \cos \phi_i & \sin \phi_i & d_i (-\sin \alpha_i \cos \phi_i + \cos \alpha_i \sin \phi_i) \end{bmatrix} \begin{bmatrix} \mathcal{R} V_{r,x} \\ \mathcal{R} V_{r,y} \\ \omega_{r,z} \end{bmatrix}$$

Next, the following trigonometric identities are applied:

$$\begin{aligned}
\sin \alpha_i \sin \phi_i + \cos \alpha_i \cos \phi_i &= \cos(\phi_i - \alpha_i) = \cos((\alpha_i + \beta_i) - \alpha_i) = \cos \beta_i \\
-\sin \alpha_i \cos \phi_i + \cos \alpha_i \sin \phi_i &= \sin(\phi_i - \alpha_i) = \sin((\alpha_i + \beta_i) - \alpha_i) = \sin \beta_i
\end{aligned}$$

Finally resulting in:

$$\begin{bmatrix} \mathcal{B}_i V_{i,x} \\ \mathcal{B}_i V_{i,y} \end{bmatrix} = \begin{bmatrix} \sin \phi_i & -\cos \phi_i & -d_i \cos \beta_i \\ \cos \phi_i & \sin \phi_i & d_i \sin \beta_i \end{bmatrix} \begin{bmatrix} \mathcal{R} V_{r,x} \\ \mathcal{R} V_{r,y} \\ \omega_{r,z} \end{bmatrix} \quad (14)$$

For each wheel i , a system of 2 linear equations with 3 unknowns ($\mathcal{R} V_{r,x}$, $\mathcal{R} V_{r,y}$, and $\omega_{r,z}$) is obtained. In general, for N wheels, $2N$ linear equations and 3 unknowns are obtained. Therefore, for $N \geq 2$, the system is over-determined. Equation 14 provides the wheel velocity expressed in its base CS $\mathcal{B}_i$. However, the physical motion constraints of the wheel originated by its contact with the ground apply in the wheel's own CS, $\mathcal{W}_i$. For this reason, it is necessary to define the intrinsic velocity components of the wheel:

- $v_{i\parallel}$: the velocity component aligned with the rolling direction (axis $X_{\mathcal{W}_i}$).
- $v_{i\perp}$: the velocity component perpendicular to the rolling direction (axis $Y_{\mathcal{W}_i}$; *i.e.*, the direction of its rotation axis).

Depending on whether the wheels used on a platform are non-steerable conventional, steerable conventional, mecanum, or omnidirectional, the geometric relationship between the components of the wheel's velocity vector $\mathbf{V}$ in the base CS (${}^{\mathcal{B}_i}V_{i,x}$, ${}^{\mathcal{B}_i}V_{i,y}$) and in the wheel-fixed CS ($v_{i\parallel}$, $v_{i\perp}$) will change. See Figure 5. Depending on the platform, different physical constraints will apply to the components $v_{i\parallel}$ and $v_{i\perp}$, yielding specific expressions that solve the forward and inverse kinematics for the considered platform.

2 General considerations on forward and inverse kinematics

In general, the forward and inverse kinematics of a terrestrial robot is defined by a matrix expression of the form:

$$A\mathbf{x} = \mathbf{b} \quad (15)$$

where:

$$A \in \mathbb{R}^{2N \times 3}, \quad \mathbf{x} \in \mathbb{R}^3, \quad \mathbf{b} \in \mathbb{R}^{2N}$$

$$A = \begin{bmatrix} \sin \phi_0 & -\cos \phi_0 & -d_0 \cos \beta_0 \\ \cos \phi_0 & \sin \phi_0 & d_0 \sin \beta_0 \\ \vdots & \vdots & \vdots \\ \sin \phi_{N-1} & -\cos \phi_{N-1} & -d_{N-1} \cos \beta_{N-1} \\ \cos \phi_{N-1} & \sin \phi_{N-1} & d_{N-1} \sin \beta_{N-1} \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} \mathcal{R}V_{r,x} \\ \mathcal{R}V_{r,y} \\ \omega_{r,z} \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} {}^{\mathcal{B}_0}V_{0,x} \\ {}^{\mathcal{B}_0}V_{0,y} \\ \vdots \\ {}^{\mathcal{B}_{N-1}}V_{N-1,x} \\ {}^{\mathcal{B}_{N-1}}V_{N-1,y} \end{bmatrix}$$

Given that for any robot with $N \geq 2$ wheels, $2N$ equations and only 3 unknowns are obtained, we are dealing with an overdetermined linear system. In a purely mathematical ideal scenario, it would suffice to take any subset of 3 linearly independent equations to solve the system and find the chassis velocity, $\mathbf{x}$. However, in the real implementation of a terrestrial robot, the equations never match perfectly due to imperfections such as:

- Noise and quantization in sensor (encoder) readings.
- Small wheel slips on the ground.
- Slight deviations in the geometric calibration of the base.

Therefore, the system is not solved by seeking an exact algebraic solution, but rather by finding the estimate $\hat{\mathbf{x}}$ that minimizes the global squared error of all wheels simultaneously:

$$\hat{\mathbf{x}} = \arg \min_{\mathbf{x}} \|\mathbf{A}\mathbf{x} - \mathbf{b}\|^2 \quad (16)$$

The solution to this least squares problem is given by the expression known as Normal Equations, which is obtained from a process of searching for the minimum of the error function (differentiating and setting to zero):

$$\mathbf{A}^\top \mathbf{A} \mathbf{x} = \mathbf{A}^\top \mathbf{b} \quad (17)$$

Therefore, the optimal solution is obtained through:

$$\hat{\mathbf{x}} = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \mathbf{b} = \mathbf{A}^+ \mathbf{b} \quad (18)$$

where $\mathbf{A}^+$ is called the Moore-Penrose pseudoinverse of matrix $\mathbf{A}$.

From a computational numerical calculation standpoint, directly inverting the matrix $(\mathbf{A}^\top \mathbf{A})$ can be unstable if the original matrix $\mathbf{A}$ is ill-conditioned. A matrix is considered ill-conditioned when small variations or noise in the input data (the vector $\mathbf{b}$) are drastically amplified, causing erratic calculations in the resulting solution $\hat{\mathbf{x}}$. The numerical misbehavior that can arise when using the normal equations formula is due to the fact that the term $(\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top$ squares the singular values of $\mathbf{A}$. If $\mathbf{A}$ has a small singular value, close to zero, squaring it makes it even smaller. When attempting to invert that value, the computer divides by a near-zero number, causing the results to spiral to infinity and all precision in the solution to be lost.

Although not strictly necessary for kinematics development, for completeness it is noted that to ascertain the geometric robustness of matrix $\mathbf{A}$, the ratio between its maximum and minimum singular values ($\kappa = \frac{\sigma_{\max}}{\sigma_{\min}}$) is used, known as the **condition number**, $\kappa(\mathbf{A})$. As a reference:

- $\kappa = 1$: $\mathbf{A}$ is the perfect matrix (example: the Identity matrix or a pure rotation matrix). There is no loss of precision.
- $1 < \kappa < 10^2$: $\mathbf{A}$ is well-conditioned. The system is robust; sensor errors will not be significantly amplified.
- $10^3 < \kappa < 10^6$: $\mathbf{A}$ is moderately ill-conditioned. Caution must be exercised, as simple methods like inverting $(\mathbf{A}^\top \mathbf{A})$ may start to yield imprecise results.
- $\kappa \approx 10^{12}$ to 10^{15} : $\mathbf{A}$ is ill-conditioned. The matrix is nearly singular (its rows/columns are almost dependent).
- $\kappa > 10^{16}$: $\mathbf{A}$ is singular and unusable. For a computer working with double precision (64 bits), this means the matrix cannot be reliably inverted.

To avoid these numerical stability issues that could arise, in practice, the pseudoinverse A^+ is obtained by applying Singular Value Decomposition (SVD) to the original matrix A . SVD is the most robust and secure numerical method, as it entirely avoids the product $(A^T A)$ and is capable of handling near-singular matrices (situations where the robot's geometric equations are so similar or redundant that the system is on the verge of not being invertible) without the resulting calculations spiraling to infinity. This method works even if matrix A does not have full rank (if some of the equations are directly contradictory).

However, it is fundamental to make a critical distinction between the software's mathematical robustness and the robot's physical viability. If, when evaluating the constant geometric matrix A during the design phase, a high condition number is obtained (for example, $\kappa > 10^3$, and critically if $\kappa > 10^6$), **it will happen that, although the SVD algorithm prevents a program crash by returning a stable mathematical solution (by truncating near-null values), in reality, a severe flaw in the mechanical design is being masked.** Physically, this means that the constructive arrangement of the wheels (their distances d_i or mounting angles α_i, β_i) generates a degenerate kinematics, where the chassis loses the ability to move independently in all its degrees of freedom, or where rolling directions directly conflict. In this scenario, even if the software delivers a seemingly valid numerical result, the physical robot will suffer uncontrollable drifts, mechanical stress, and erratic movements at the slightest noise in the encoders. Therefore, SVD should not be used as an excuse to ignore bad geometry; if the initial condition number is excessively high, the inevitable solution involves redesigning the wheel placement and base orientation in the CAD design until a well-conditioned system is achieved (ideally $\kappa \leq 100$).

SVD decomposes the matrix into $A = U\Sigma V^T$, allowing the pseudoinverse to be constructed directly as $A^+ = V\Sigma^+ U^T$.

Proof:

If $A = U\Sigma V^T$, the transpose property for a matrix product is applied: $(XYZ)^T = Z^T Y^T X^T$ (the order is reversed).

$$A^T = (V^T)^T \Sigma^T U^T \quad (19)$$

Transposing twice returns the original matrix $((V^T)^T = V)$, and since Σ is a diagonal matrix that does not change upon transposing $(\Sigma^T = \Sigma)$ ¹, we

¹For the equality $\Sigma^T = \Sigma$ to be strictly true, Σ must be a square matrix. In over-determined systems, where A is a rectangular $M \times N$ matrix, the original Σ matrix is rectangular $M \times N$ and Σ^T would be $N \times M$, hence the equality would not hold. However, in practice, the Reduced SVD (or *Thin SVD*) is typically employed to solve these systems,

obtain:

$$A^T = V\Sigma U^T \quad (20)$$

Substituting into the product $A^T A$:

$$A^T A = (V\Sigma U^T)(U\Sigma V^T) \quad (21)$$

By the associative property of matrices, the inner parentheses can be omitted:

$$A^T A = V\Sigma(U^T U)\Sigma V^T \quad (22)$$

Matrix U is orthogonal, meaning its transpose multiplied by itself results in the Identity matrix ($U^T U = I$):

$$A^T A = V\Sigma I \Sigma V^T \quad (23)$$

Multiplying by the Identity matrix does not alter the result, and Σ multiplied by itself is Σ^2 (which is equivalent to squaring the values of its diagonal):

$$A^T A = V\Sigma^2 V^T \quad (24)$$

Next, the inverse is applied to the entire expression using the inverse property for a product: $(XYZ)^{-1} = Z^{-1}Y^{-1}X^{-1}$.

$$(A^T A)^{-1} = (V^T)^{-1}(\Sigma^2)^{-1}V^{-1} \quad (25)$$

Since V is also an orthogonal matrix, its inverse is equal to its transpose, $V^{-1} = V^T$, and thus, $(V^T)^{-1} = V$. The inverse of a diagonal matrix Σ^2 is simply denoted as Σ^{-2} (which consists of inverting the squared values of its diagonal). Substituting these properties yields:

$$(A^T A)^{-1} = V\Sigma^{-2}V^T \quad (26)$$

It is now multiplied by A^T from the right to complete the Normal Equations expression:

which trims the rows of zeros from the Σ matrix, converting it into a square $N \times N$ matrix. In this way, $\Sigma^T = \Sigma$ does hold in the Reduced SVD, allowing the formula $A^+ = V\Sigma^+U^T$ to be developed without dimensional conflicts. In the Eigen library for C++, combining the options `ComputeThinU` | `ComputeThinV` performs precisely this calculation.

$$(A^T A)^{-1} A^T = (V \Sigma^{-2} V^T)(V \Sigma U^T) \quad (27)$$

The inner parentheses are removed again via the associative property:

$$(A^T A)^{-1} A^T = V \Sigma^{-2} (V^T V) \Sigma U^T \quad (28)$$

Once more, given that V is orthogonal, $V^T V = I$. The central term disappears:

$$(A^T A)^{-1} A^T = V \Sigma^{-2} \Sigma U^T \quad (29)$$

At this point, we have Σ^{-2} multiplied by Σ . Because both are diagonal matrices, the operation is equivalent to multiplying each element of the Σ^{-2} diagonal by the corresponding element of the Σ diagonal. So each element of the resulting diagonal is the original Σ value raised to $-2 + 1 = -1$ (that is, the reciprocal of the original Σ values). This resulting diagonal matrix is denoted as Σ^+ :

- If A does **not** have full rank, Σ^+ is defined as the diagonal matrix containing the reciprocals of the non-zero singular values of A , and zeros in the positions corresponding to the zero singular values.
- If A **has** full rank, then Σ^+ is actually Σ^{-1} (a proper inverse), which is the diagonal matrix containing the reciprocals of all the singular values of A .

Finally, substituting Σ^+ , the expression is reached:

$$(A^T A)^{-1} A^T = V \Sigma^+ U^T \quad (30)$$

As previously discussed, the computational problem arises when the computer directly evaluates the expression $(A^T A)^{-1}$, since it must square singular values that could be very close to zero. When attempting to invert those nearly zero squared values, numerical overflows occur. By using the direct final formula $V \Sigma^+ U^T$ provided by SVD, the algorithm only inverts the original values (Σ^+), keeping the system stable at all times.

3 Kinematics of a three swerve drive robot

The base with three steerable active wheels (three swerve drive) is an omnidirectional locomotion system. It allows a terrestrial robot to move in any direction without first having to change the chassis orientation. Each wheel is steered independently, so the robot gains maneuverability and flexibility.

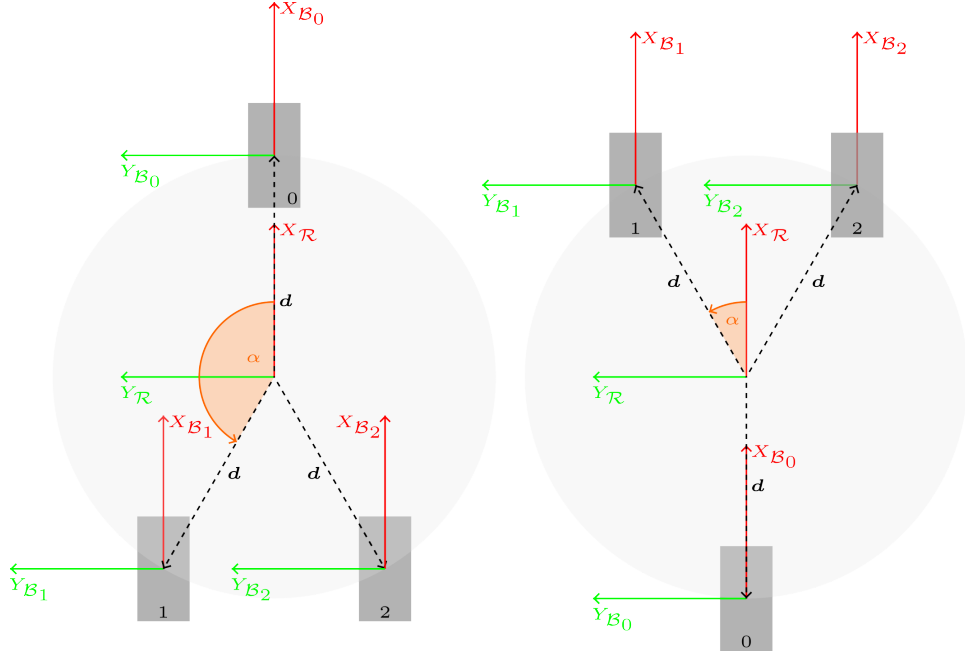


Figure 6: Common wheel configurations in a three swerve drive system (left: inverted-Y configuration, right: Y configuration)

All wheels have the same radius r and are located at the same distance d from the center of the base. Furthermore, in this particular case, it is imposed:

$$\begin{aligned}\psi_i &= \phi_i - \frac{\pi}{2} = 0 \implies \phi_i = \frac{\pi}{2} \\ \phi_i &= \alpha_i + \beta_i = \frac{\pi}{2}, \implies \beta_i = \frac{\pi}{2} - \alpha_i\end{aligned}$$

and therefore:

$${}^{\mathcal{B}_i}R\left(\phi_i = \frac{\pi}{2}\right)_{\mathcal{R}} = \begin{bmatrix} \sin \frac{\pi}{2} & -\cos \frac{\pi}{2} \\ \cos \frac{\pi}{2} & \sin \frac{\pi}{2} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

For the inverted-Y configuration:

- $\alpha \in [\frac{\pi}{2}, \pi)$ and $d > 0$
- Front wheel ($i = 0$):

$$\begin{aligned}\alpha_0 &= 0 \\ \mathbf{d}_0 &= (d \cos 0, d \sin 0) = (d, 0) \\ \beta_0 &= \frac{\pi}{2} - 0 = \frac{\pi}{2}\end{aligned}$$

- Left wheel ($i = 1$):

$$\begin{aligned}\alpha_1 &= \alpha \\ \mathbf{d}_1 &= (d \cos \alpha, d \sin \alpha) \\ \beta_1 &= \frac{\pi}{2} - \alpha\end{aligned}$$

- Right wheel ($i = 2$):

$$\begin{aligned}\alpha_2 &= -\alpha \\ \mathbf{d}_2 &= (d \cos(-\alpha), d \sin(-\alpha)) = (d \cos \alpha, -d \sin \alpha) \\ \beta_2 &= \frac{\pi}{2} - (-\alpha) = \frac{\pi}{2} + \alpha\end{aligned}$$

The matrix A for the inverted-Y configuration is presented below:

$$\begin{aligned}A &= \begin{bmatrix} \sin \phi_0 & -\cos \phi_0 & -d_0 \cos \beta_0 \\ \cos \phi_0 & \sin \phi_0 & d_0 \sin \beta_0 \\ \sin \phi_1 & -\cos \phi_1 & -d_1 \cos \beta_1 \\ \cos \phi_1 & \sin \phi_1 & d_1 \sin \beta_1 \\ \sin \phi_2 & -\cos \phi_2 & -d_2 \cos \beta_2 \\ \cos \phi_2 & \sin \phi_2 & d_2 \sin \beta_2 \end{bmatrix} = \begin{bmatrix} 1 & 0 & -d \cos(\frac{\pi}{2} - 0) \\ 0 & 1 & d \sin(\frac{\pi}{2} - 0) \\ 1 & 0 & -d \cos(\frac{\pi}{2} - \alpha) \\ 0 & 1 & d \sin(\frac{\pi}{2} - \alpha) \\ 1 & 0 & -d \cos(\frac{\pi}{2} - (-\alpha)) \\ 0 & 1 & d \sin(\frac{\pi}{2} - (-\alpha)) \end{bmatrix} = \\ &= \begin{bmatrix} 1 & 0 & -d \sin 0 \\ 0 & 1 & d \cos 0 \\ 1 & 0 & -d \sin \alpha \\ 0 & 1 & d \cos \alpha \\ 1 & 0 & -d \sin(-\alpha) \\ 0 & 1 & d \cos(-\alpha) \end{bmatrix} = \begin{bmatrix} 1 & 0 & -y_0 \\ 0 & 1 & x_0 \\ 1 & 0 & -y_1 \\ 0 & 1 & x_1 \\ 1 & 0 & -y_2 \\ 0 & 1 & x_2 \end{bmatrix} \quad (31)\end{aligned}$$

For the Y configuration:

- $\alpha \in (0, \frac{\pi}{2}]$ and $d > 0$
- Rear wheel ($i = 0$):

$$\begin{aligned}\alpha_0 &= \pi \\ \mathbf{d}_0 &= (d \cos \pi, d \sin \pi) = (-d, 0) \\ \beta_0 &= \frac{\pi}{2} - \pi = -\frac{\pi}{2}\end{aligned}$$

- Left wheel ($i = 1$):

$$\begin{aligned}\alpha_1 &= \alpha \\ \mathbf{d}_1 &= (d \cos \alpha, d \sin \alpha) \\ \beta_1 &= \frac{\pi}{2} - \alpha\end{aligned}$$

- Right wheel ($i = 2$):

$$\begin{aligned}\alpha_2 &= -\alpha \\ \mathbf{d}_2 &= (d \cos(-\alpha), d \sin(-\alpha)) = (d \cos \alpha, -d \sin \alpha) \\ \beta_2 &= \frac{\pi}{2} - (-\alpha) = \frac{\pi}{2} + \alpha\end{aligned}$$

The matrix A for the Y configuration is presented below:

$$\begin{aligned}A &= \begin{bmatrix} \sin \phi_0 & -\cos \phi_0 & -d_0 \cos \beta_0 \\ \cos \phi_0 & \sin \phi_0 & d_0 \sin \beta_0 \\ \sin \phi_1 & -\cos \phi_1 & -d_1 \cos \beta_1 \\ \cos \phi_1 & \sin \phi_1 & d_1 \sin \beta_1 \\ \sin \phi_2 & -\cos \phi_2 & -d_2 \cos \beta_2 \\ \cos \phi_2 & \sin \phi_2 & d_2 \sin \beta_2 \end{bmatrix} = \begin{bmatrix} 1 & 0 & -d \cos(\frac{\pi}{2} - \pi) \\ 0 & 1 & d \sin(\frac{\pi}{2} - \pi) \\ 1 & 0 & -d \cos(\frac{\pi}{2} - \alpha) \\ 0 & 1 & d \sin(\frac{\pi}{2} - \alpha) \\ 1 & 0 & -d \cos(\frac{\pi}{2} - (-\alpha)) \\ 0 & 1 & d \sin(\frac{\pi}{2} - (-\alpha)) \end{bmatrix} = \\ &= \begin{bmatrix} 1 & 0 & -d \sin \pi \\ 0 & 1 & d \cos \pi \\ 1 & 0 & -d \sin \alpha \\ 0 & 1 & d \cos \alpha \\ 1 & 0 & -d \sin(-\alpha) \\ 0 & 1 & d \cos(-\alpha) \end{bmatrix} = \begin{bmatrix} 1 & 0 & -y_0 \\ 0 & 1 & x_0 \\ 1 & 0 & -y_1 \\ 0 & 1 & x_1 \\ 1 & 0 & -y_2 \\ 0 & 1 & x_2 \end{bmatrix} \quad (32)\end{aligned}$$

Therefore, it is demonstrated that both configurations, inverted-Y and Y, share the same matrix A , which implies that their forward and inverse kinematics are identical. The only difference between both configurations lies in the physical arrangement of the wheels, but kinematically, both are equivalent.

3.1 Inverse kinematics of the three swerve

For a steerable conventional wheel, the following holds:

$$\omega_i \mathbf{V}_i = \begin{bmatrix} v_{i\parallel} \\ v_{i\perp} \end{bmatrix} = \begin{bmatrix} \omega_i r \\ 0 \end{bmatrix} \quad (33)$$

where $v_{i\parallel}$ is the velocity of wheel i in the rolling direction and $v_{i\perp}$ is the component perpendicular to the rolling direction, which is zero, since the wheel cannot slip laterally. Given that the steerable wheel must rotate by an angle θ_i relative to its base CS $\mathcal{B}_i$ to align its rolling direction (axis $X_{\mathcal{W}_i}$) with the wheel velocity vector $\mathbf{V}_i$, it is evident that:

$$v_{i\parallel} = \|\mathcal{B}_i \mathbf{V}_i\| = \sqrt{\mathcal{B}_i V_{i,x}^2 + \mathcal{B}_i V_{i,y}^2} = r\omega_i \quad (34)$$

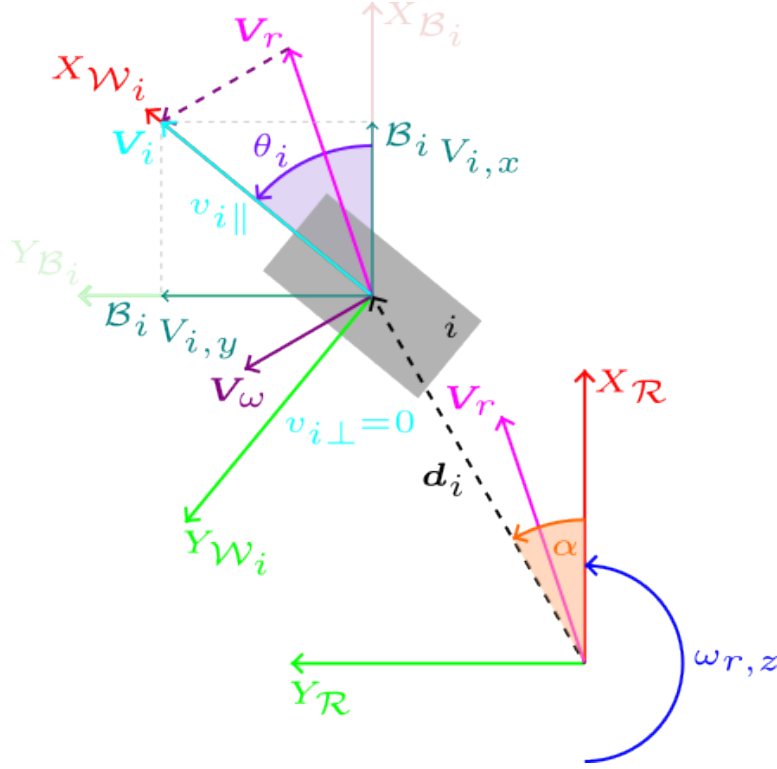


Figure 7: Steerable conventional wheel aligned with its velocity vector $\mathbf{V}_i$ via a rotation of angle θ_i relative to its base CS $\mathcal{B}_i$. The rolling direction is defined by the $X_{\mathcal{W}_i}$ axis of the CS fixed to wheel i , $\mathcal{W}_i$.

Steps to calculate the inverse kinematics of a robot with a three swerve drive base:

- Given the chassis velocity vector ${}^{\mathcal{R}}\mathbf{V}_r$, the velocity of each wheel i in its base CS $\mathcal{B}_i$, ${}^{\mathcal{B}_i}\mathbf{V}_i$, is obtained by applying the coordinate transformation of Equation 14.
- The steering angle θ_i that each wheel i must adopt to align its rolling direction with the velocity vector ${}^{\mathcal{B}_i}\mathbf{V}_i$ is calculated using the arctan 2 function.

$$\theta_i = \arctan 2({}^{\mathcal{B}_i}V_{i,y}, {}^{\mathcal{B}_i}V_{i,x}) \quad (35)$$

- The angular velocity of each wheel i is calculated from Expression 34.

$$\omega_i = \frac{v_{i||}}{r} = \frac{\|{}^{\mathcal{B}_i}\mathbf{V}_i\|}{r} = \frac{\sqrt{{}^{\mathcal{B}_i}V_{i,x}^2 + {}^{\mathcal{B}_i}V_{i,y}^2}}{r} \quad (36)$$

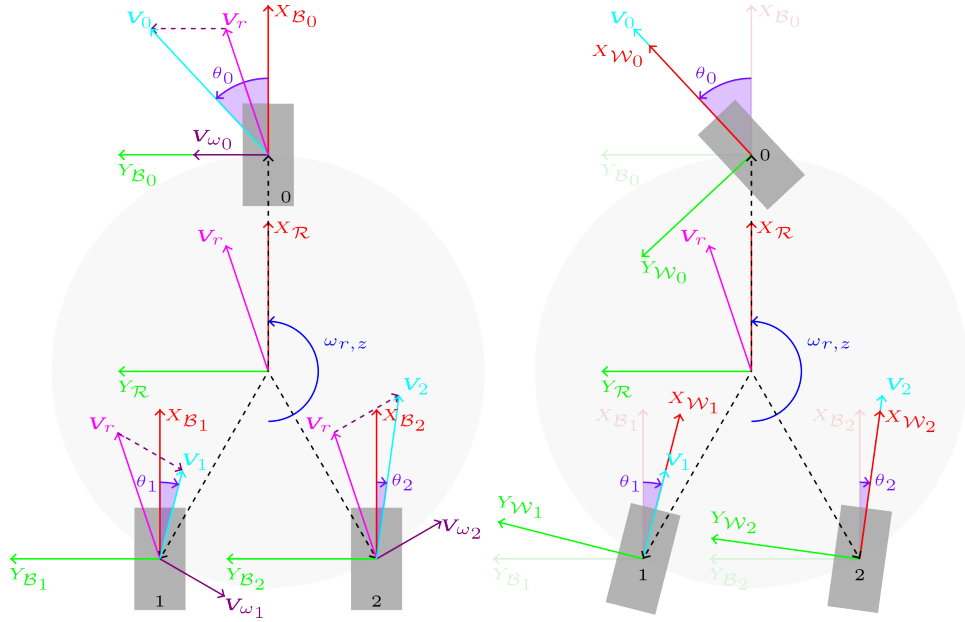


Figure 8: Inverted-Y configuration. Left: Calculation of the velocity vector for each wheel, V_i . Right: Final orientation that each wheel must adopt (θ_i) to align its rolling direction with the velocity vector V_i .

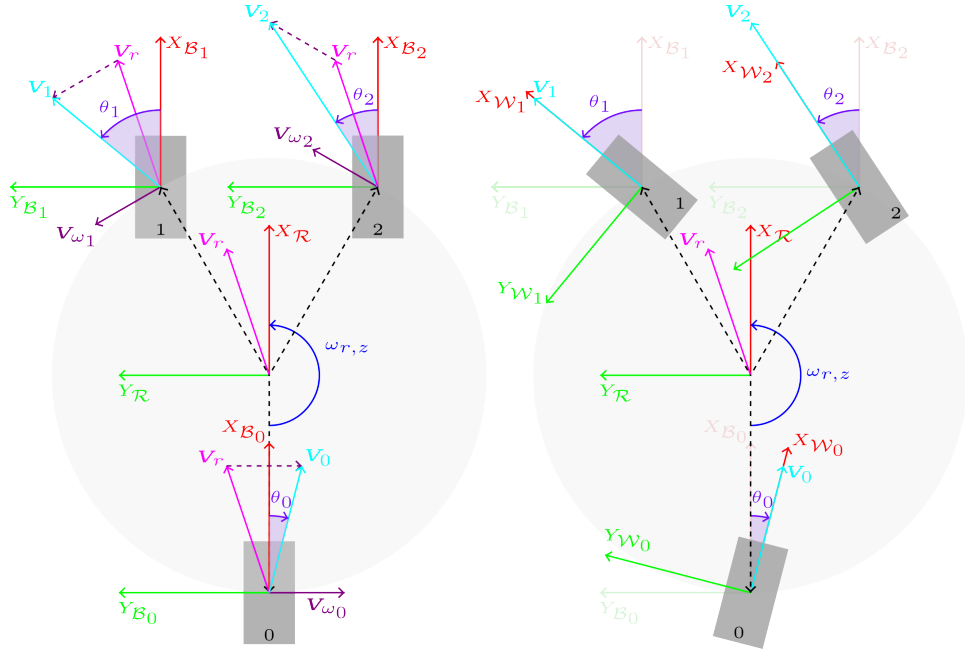


Figure 9: Y configuration. Left: Calculation of the velocity vector for each wheel, V_i . Right: Final orientation that each wheel must adopt (θ_i) to align its rolling direction with the velocity vector V_i .

3.2 Forward kinematics of the three swerve

Known the steering angle θ_i and the angular velocity ω_i of each wheel i , the following steps are executed to calculate the forward kinematics of a robot with a three swerve drive base:

- Calculate the vector $\mathbf{b} = [\mathcal{B}_0 V_{0,x}, \mathcal{B}_0 V_{0,y}, \dots, \mathcal{B}_{N-1} V_{N-1,x}, \mathcal{B}_{N-1} V_{N-1,y}]^T$.

Where:

$$\begin{aligned} \mathcal{B}_i V_{i,x} &= \|\mathcal{B}_i \mathbf{V}_i\| \cos \theta_i = v_i \cos \theta_i = r \omega_i \cos \theta_i \\ \mathcal{B}_i V_{i,y} &= \|\mathcal{B}_i \mathbf{V}_i\| \sin \theta_i = v_i \sin \theta_i = r \omega_i \sin \theta_i \end{aligned} \quad (37)$$

- Solve the linear system $A\mathbf{x} = \mathbf{b}$ using the pseudoinverse of A , A^+ , as explained in Section 2, to obtain the chassis velocity $\mathbf{x} = [\mathcal{R} V_{r,x}, \mathcal{R} V_{r,y}, \omega_{r,z}]^T$. Obviously, using the matrix A corresponding to the considered wheel configuration, inverted-Y or Y.